FORMATION « PYTHON EN 24 HEURES » | SÉANCE 07

Fonctions avancées : closures, décorateurs, chaînes et fichiers

Chapitres 13 et 14 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Comprendre la portée LEGB et l'imbrication de fonctions
- Écrire des closures : des fonctions qui gardent en mémoire leur contexte
- Comprendre et écrire un décorateur avec @
- Maîtriser les méthodes essentielles des chaînes : strip, lower, split, join
- Lire et écrire des fichiers proprement avec with open

Rappel de la séance 06

Une fonction est un objet comme un autre : on peut la retourner, la passer en paramètre et même la fabriquer à la chaîne. C'est la clé des closures.

- Dict : accès par clé, .items(), schéma de comptage
- Set : unicité, appartenance rapide, union et intersection
- Fonctions : def, paramètres, valeurs par défaut, return
- Portée : une variable créée dans une fonction disparaît à la sortie
- Choisir la structure : position ? list. Nom ? dict. Unicité ? set

La règle LEGB : où Python cherche un nom

Niveau	Signification	Durée de vie
L	Local : dans la fonction courante	à chaque appel
E	Englobante : la fonction qui contient	tant que la closure vit
G	Globale : au niveau du module	toute l'exécution
B	Built in : print, len, sum...	toujours

Python cherche de bas en haut et s'arrête au premier nom trouvé. Une affectation crée un nom LOCAL, sauf déclaration nonlocal ou global.

Fonctions imbriquées

```
def exterior(x):
    def interieur(y):
        return x + y       # x vient de la portée englobante
    return interieur

ajoute_10 = exterior(10)
print(ajoute_10(5))        # 15
print(ajoute_10(32))       # 42
```

La fonction interne peut lire les variables de la fonction externe. Retourner ‘interieur‘ exporte la fonction, avec son souvenir de x.

La closure : la fonction à mémoire

- Une **closure** est une fonction interne retournée qui **conserve** les variables de sa portée de naissance
- Chaque appel de la fabrique crée une closure **indépendante**
- ‘ajoute_10.__closure__‘ révèle la valeur retenue
- Usage : fabriques de fonctions configurées, compteurs, caches
- C’est le socle des décorateurs, vus dans trois diapositives

Fabrique de multiplicateurs

```
def fabrique_multiplicateur(factor):
    def multiplier(valeur):
        return valeur * factor    # factor est retenu
    return multiplier

double = fabrique_multiplicateur(2)
triple = fabrique_multiplicateur(3)
print(double(7), triple(7))    # 14 21
```

double et triple sont deux closures distinctes : chacune a retenu son propre factor. Changer double ne touche pas triple.

Modifier la variable englobante : nonlocal

- Lire la variable englobante : automatique
- La **réaffecter** : il faut déclarer 'nonlocal compteur'
- Sans nonlocal, l'affectation créerait une variable locale de même nom
- Schéma classique : le compteur vit dans la closure, personne d'autre ne peut y toucher

```
def fabrique_compteur():
    compteur = 0
    def incrementer():
        nonlocal compteur
        compteur += 1
        return compteur
    return incrementer

prochain = fabrique_compteur()
print(prochain(), prochain(), prochain())
print(prochain.__closure__[0].cell_contents)
```

Le décorateur : une closure appliquée

```python
def majuscules(fonction):
    def emballage(*args, **kwargs):
        resultat = fonction(*args, **kwargs)
        return resultat.upper()
    return emballage

@majuscules
def salutation(nom):
    return f"bonjour {nom}"

print(salutation("Awa"))    # BONJOUR AWA
```

@majuscules est un raccourci pour : salutation = majuscules(salutation). La fonction d'origine vit toujours, emballée.

Préserver l'identité : functools.wraps

- Après emballage, 'salutation.__name__' vaudrait « emballage »
- @wraps(fonction) recopie nom, docstring et métadonnées
- Indispensable pour le débogage et la documentation
- Les arguments *args et **kwargs rendent le décorateur universel

```
from functools import wraps

def compte_appels(fonction):
    @wraps(fonction)
    def emballage(*args, **kwargs):
        emballage.appels += 1
        return fonction(*args, **kwargs)
    emballage.appels = 0
    return emballage
```

Les méthodes des chaînes

```
phrase = "  Bonjour la Classe  "
print(phrase.strip())             # retire les espaces aux bords
print(phrase.strip().lower())     # tout en minuscules
mots = "riz,yam,huile".split(",")
print(mots)                       # ['riz', 'yam', 'huile']
print(" + ".join(mots))           # riz + yam + huile
print("abc".replace("a", "X"))    # Xbc
```

Les chaînes sont immuables : chaque méthode retourne une NOUVELLE chaîne. Enchaînez les comme des filtres : phrase.strip().lower().split().

Fichiers : la règle with open

- ‘open(nom, mode)‘ ouvre : « r » lire, « w » écrire en écrasant, « a » ajouter à la fin
- **with** ferme le fichier même en cas d’erreur : jamais de fichier oublié
- ‘.read()‘ lit tout, ‘.readlines()‘ une liste de lignes, boucler lit ligne à ligne
- Chaque ligne lue se termine par \{}n : ‘strip()‘ la nettoie
- On détaillera les erreurs de fichier à la séance 11

```
with open("notes.txt", "w") as f:
    f.write("12\n15\n9\n")

with open("notes.txt", "r") as f:
    total = 0
    for ligne in f:
        total += int(ligne.strip())
print(total)   # 36
```

Astuce

Explorateur de closure

Après avoir créé une closure, inspectez la : ‘ma_fonction.__closure__‘ est un tuple de cellules, et ‘.cell_contents‘ montre la valeur retenue. Ce microscope rend concret un concept souvent abstrait : la fonction garde bien une boîte avec la variable dedans.

Attention

w écrase, a ajoute

Ouvrir un fichier en mode « w » efface immédiatement tout son contenu, avant même la première écriture. Pour enrichir un journal au fil des jours, utilisez « a ». Et rappelez vous que les chaînes sont immuables : ‘phrase.upper()‘ ne modifie rien si vous n’affectez pas le résultat.

À retenir

L'essentiel de la séance 07

LEGB régit la recherche des noms ; la closure retient la portée englobante et nonlocal autorise sa modification ; le décorateur enveloppe une fonction pour lui ajouter un comportement. Les chaînes se traitent par chaînage de méthodes immuables. Les fichiers se manipulent toujours avec with open : lire, écrire, ajouter.

et fichiers

Cinq exercices qui font dialoguer fonctions avancées et données textuelles. Le dernier exercice écrit réellement un fichier sur le disque, puis le supprime proprement.

- **Exercice 1 (Basique)** : Nettoyage de chaîne
- **Exercice 2 (Basique)** : Palindrome
- **Exercice 3 (Intermédiaire)** : Fabrique de puissances (closure)
- **Exercice 4 (Intermédiaire)** : Décorateur compteur d'appels
- **Exercice 5 (Défi)** : Journal de bord dans un fichier

Ouvrez le notebook « Seance_07_closures_decorateurs_fichiers.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 08 : la programmation orientée objet. Tout ce que vous avez écrit jusqu'ici était procédural ; nous allons ranger données et comportements dans des classes. Relisez le schéma de comptage : la classe Compte en aura besoin pour son historique.